



**Università
di Catania**

Multimedia Forensics – 6 CFU

Computer Vision – 9 CFU

A.A. 2025/2026

**Corso di Laurea Magistrale in
Informatica**

**Dipartimento di Matematica e
Informatica**

Relazione Progetto

**Localizzazione di manipolazioni in immagini,
estrazione degli oggetti manipolati e
riconoscimento di istanze tramite metric
learning**

Mannino Daniele – 1000015345

Codice eseguibile, relazione e risultati dei report presenti sul repository git:

[https://github.com/DanyMannino22/Progetto MF CV](https://github.com/DanyMannino22/Progetto_MF_CV)

Indice

Indice	2
1. Introduzione e obiettivi del progetto	3
1.1 Struttura della pipeline	3
2. Dataset e analisi esplorativa (EDA)	4
2.1 Suddivisione del dataset	4
2.2 Statistiche esplorative sul training set	4
Dimensioni delle immagini	4
Binarieta' delle maschere	4
Area manipolata (class imbalance)	5
3. Fase 1 - Modello di segmentazione (manipulation localization)	6
3.1 Architettura	6
3.2 Dataset e preprocessing	6
3.3 Loss function	7
3.4 Setup di training	7
3.5 Risultati - Fase A: encoder congelato	8
3.6 Risultati - Fase B: fine-tuning con encoder sbloccato	8
3.7 Valutazione finale sul test set	9
4. Fase 2 - Estrazione degli oggetti manipolati	10
4.1 Estrazione da ground truth (dataset di riferimento)	10
4.2 Estrazione da maschere predette (pipeline reale)	10
5. Fase 3 - Metric learning per il riconoscimento di istanze	12
5.1 Motivazione e scelta metodologica	12
5.2 Architettura	12
5.3 Loss: NT-Xent (contrastive, stile SimCLR)	13
5.4 Difficolta' tecnica: batch size e potenza di calcolo	13
5.5 Risultati del training	13
5.6 Validazione qualitativa dello spazio di embedding	14
6. Applicazione: ricerca di raggruppamenti nell'intero dataset	15
6.1 Primo tentativo: DBSCAN con soglia 0.7 - problema di chaining	15
6.2 Iterazioni successive	15
6.3 Persistenza della galleria di embedding	16
7. Pipeline di inferenza end-to-end	17
7.1 Guida pratica: come eseguire l'inferenza su una nuova immagine	17
Passo 1 - Generare (o aggiornare) la galleria di riferimento	17
Passo 2 - Eseguire l'inferenza su una nuova immagine	17
Passo 3 - Interpretare l'output	18
Parametri configurabili	18
8. Conclusioni, limiti e sviluppi futuri	19
8.1 Risultati raggiunti	19
8.2 Limiti riscontrati	19
8.3 Sviluppi futuri	19

1. Introduzione e obiettivi del progetto

Il progetto affronta il problema della manipulation detection e localization su immagini digitali, con l'obiettivo di costruire una pipeline completa in tre fasi: (1) individuare automaticamente, tramite un modello di segmentazione semantica, la regione di un'immagine che è stata manipolata; (2) estrarre l'oggetto manipolato dall'immagine a partire dalla maschera predetta; (3) confrontare gli oggetti estratti tra loro tramite un modello di metric learning, per determinare se rappresentano la stessa istanza fisica riutilizzata in più immagini del dataset.

Il dataset di partenza è composto da 5000 coppie immagine/maschera di ground truth. Ogni immagine contiene una manipolazione (splicing, copy-move, inpainting via diffusion model, face morphing/swapping, ecc.) e la relativa maschera binaria bianco/nero che ne evidenzia la regione alterata.

1.1 Struttura della pipeline

- Fase 1 - Segmentazione: rete U-Net con encoder ResNet pretrained per predire la maschera di manipolazione.
- Fase 2 - Estrazione oggetti: calcolo del bounding box dalla maschera (GT o predetta) e ritaglio dell'oggetto dall'immagine originale.
- Fase 3 - Metric learning: rete Siamese allenata con instance discrimination (contrastive learning, NT-Xent loss) per generare embedding che permettano di riconoscere se due oggetti estratti sono la stessa istanza.
- Fase 4 - Inferenza end-to-end: pipeline completa che, data una nuova immagine, applica in sequenza i tre modelli e confronta il risultato con una galleria di embedding di riferimento.

Nota metodologica

In diversi punti del progetto sono emersi risultati subottimali (overfitting nel fine-tuning della segmentazione, un problema di chaining nel clustering degli embedding) che sono stati diagnosticati e corretti nel corso del lavoro. Questa relazione documenta anche questi passaggi, poiché rappresentano un contributo metodologico importante quanto i risultati positivi.

2. Dataset e analisi esplorativa (EDA)

2.1 Suddivisione del dataset

Le 5000 coppie immagine/maschera sono state suddivise in modo casuale (seed fisso per riproducibilità) in:

Split	Numero di coppie	Percentuale
Train	4000	80%
Validation	500	10%
Test	500	10%

Lo split è stato eseguito allineando i nomi file tra le cartelle images/ e masks/, scartando eventuali file orfani (senza controparte) prima dello shuffle.

2.2 Statistiche esplorative sul training set

L'analisi del set di training (4000 immagini) ha prodotto le seguenti osservazioni chiave:

Dimensioni delle immagini

Il dataset presenta un'elevata eterogeneità dimensionale: 254 dimensioni uniche su 4000 immagini. Le cinque dimensioni più frequenti coprono solo il 66% del totale:

Dimensione (WxH)	Numero immagini
256x256	896
500x652	740
512x512	658
640x480	214
880x1184	156

Questa variabilità ha reso necessario un resize a dimensione fissa nel Dataset PyTorch, con interpolazione differenziata per immagine (bicubica) e maschera (nearest-neighbor, per non introdurre valori intermedi).

Binarietà delle maschere

1953 maschere su 4000 (48.8%) non risultavano strettamente binarie (0/255), a causa di antialiasing sui bordi della regione manipolata. Questo ha richiesto una binarizzazione esplicita (soglia > 127) applicata dopo il resize, per garantire target puliti alla loss.

Area manipolata (class imbalance)

Statistica	Valore
Media	0.1761
Mediana	0.1580
Minimo	0.0001
Massimo	0.9967
Deviazione standard	0.1460
Maschere completamente vuote	0 (nessuna)

La percentuale di pixel manipolati varia moltissimo da immagine a immagine (da 0.01% a 99.67%), con una media intorno al 17.6%. Questo sbilanciamento moderato ma altamente variabile per-immagine ha motivato la scelta di una loss combinata BCE + Dice (Sezione 3.3), più robusta di un semplice class-weighting fisso quando la scala del problema cambia così tanto da un caso all'altro.

3. Fase 1 - Modello di segmentazione (manipulation localization)

3.1 Architettura

È stata scelta un'architettura U-Net, implementata tramite la libreria `segmentation_models_pytorch`, con le seguenti caratteristiche:

- Encoder: ResNet18 pretrained su ImageNet.
- Decoder: implementazione U-Net standard della libreria, con skip connection tra encoder e decoder.
- Output: logits grezzi a singolo canale (nessuna sigmoid nel modello), per compatibilità numerica con BCEWithLogitsLoss.
- Strategia di training a due fasi: (a) encoder congelato (feature extractor puro) per un primo training rapido del solo decoder; (b) fine-tuning completo con encoder sbloccato e learning rate differenziato.

Un dettaglio implementativo importante riguarda il congelamento dell'encoder: non è sufficiente impostare `requires_grad=False` sui pesi, perché le statistiche di BatchNorm (running mean/var) continuerebbero ad aggiornarsi durante il training. È stato quindi fatto l'override del metodo `train()` del modello:

```
def build_model(encoder_name, encoder_weights, freeze_encoder):
    model = smp.Unet(encoder_name=encoder_name,
                     encoder_weights=encoder_weights,
                     in_channels=3, classes=1, activation=None)
    if freeze_encoder:
        for param in model.encoder.parameters():
            param.requires_grad = False
        model._freeze_encoder = freeze_encoder

    original_train = model.train
    def train_override(self, mode=True):
        original_train(mode)
        if getattr(self, '_freeze_encoder', False):
            self.encoder.eval() # BatchNorm resta congelato
        return self
    model.train = train_override.__get__(model)
    return model
```

3.2 Dataset e preprocessing

- Resize a 512x512 (bicubico per l'immagine, nearest per la maschera + binarizzazione post-resize a soglia 127).
- Normalizzazione ImageNet (mean/std standard), necessaria per compatibilità con l'encoder pretrained.

- Augmentation in training (albugmentations): flip orizzontale/verticale, rotazioni di 90 gradi, shift/scale/rotate leggero, variazioni di luminosita'/contrasto/hue.
- Nessuna augmentation in validation/test, solo resize deterministico e normalizzazione.

3.3 Loss function

È stata utilizzata una combinazione pesata (50/50) di Binary Cross-Entropy e Dice Loss:

```
class BCDiceLoss(nn.Module):
    def __init__(self, bce_weight=0.5):
        super().__init__()
        self.bce_weight = bce_weight
        self.bce = nn.BCEWithLogitsLoss()

    def forward(self, logits, targets):
        bce_loss = self.bce(logits, targets)
        probs = torch.sigmoid(logits)
        dice_loss = 1.0 - dice_coefficient(probs, targets)
        return self.bce_weight * bce_loss + (1 - self.bce_weight) * dice_loss
```

La componente BCE stabilizza il training pixel-per-pixel; la componente Dice gestisce meglio lo sbilanciamento di scala osservato nell'EDA (Sezione 2.2), risultando più robusta della sola BCE quando l'area manipolata varia così tanto da immagine a immagine.

3.4 Setup di training

- Ottimizzatore: AdamW, applicato solo ai parametri allenabili (encoder escluso nella prima fase).
- Mixed precision (AMP) per velocizzare il training su GPU.
- Scheduler ReduceLROnPlateau (dimezza il LR se il Dice di validation non migliora per 5 epoche).
- Early stopping con patience maggiore della patience dello scheduler, per garantire che il LR abbia la possibilità di scendere prima che il training si interrompa.
- Checkpoint best/last salvati a ogni epoca, in base al miglior Dice di validation.

3.5 Risultati - Fase A: encoder congelato

Il training con encoder congelato (50 epoche configurate, learning rate $1e-4$) è stato interrotto dall'early stopping all'epoca 21, dopo aver raggiunto il miglior risultato all'epoca 16:

Epoca	Train Loss	Train Dice	Val Loss	Val Dice
1	0.5579	0.3348	0.4679	0.4152
8	0.4286	0.4541	0.4223	0.4810
16 (best)	0.3933	0.4989	0.4264	0.5011
21 (stop)	0.3756	0.5221	0.4208	0.4919

Il training è risultato stabile (nessun segno di overfitting: la val loss non è mai risalita in modo marcato), ma con una convergenza lenta dovuta alla capacità limitata del solo decoder allenabile. Miglior Dice di validation ottenuto: 0.5011.

3.6 Risultati - Fase B: fine-tuning con encoder sbloccato

Ripartendo dal checkpoint della Fase A, l'encoder è stato sbloccato e allenato con learning rate differenziato: $1e-5$ per l'encoder (pretrained, da aggiornare con cautela), $1e-4$ per il decoder.

Epoca	Train Dice	Val Dice	Train Loss	Val Loss
1	0.4661	0.4858	0.4222	0.4084
4	0.5040	0.5057	0.3888	0.4003
7	0.5274	0.5192	0.3712	0.4033
19 (best)	0.6179	0.5260	0.2999	0.4191
30 (fine)	0.6599	0.5137	0.2657	0.4251

Osservazione critica: overfitting

Dall'epoca ~13 in poi il train Dice continua a salire fino a 0.66, mentre il val Dice oscilla senza un reale trend di miglioramento tra 0.49 e 0.53, e la val loss inizia lievemente a risalire (da 0.40 verso 0.42-0.44). Questo è un chiaro segnale di overfitting: il modello inizia a specializzarsi sul training set senza guadagni di generalizzazione. Il gap train/val Dice a fine training è di circa 0.15 punti (0.66 vs 0.51).

Il miglior modello (epoca 19, val Dice 0.5260) è stato conservato come checkpoint definitivo, in linea con la buona pratica di selezionare il modello in base alla metrica di validation e non all'ultima epoca disponibile. Il miglioramento rispetto alla Fase A è modesto (+2.5 punti percentuali di Dice) ma reale.

3.7 Valutazione finale sul test set

La valutazione finale (eseguita una sola volta, a modello già selezionato tramite validation, per non introdurre data leakage) ha dato:

Metrica	Validation	Test
Loss	0.4191	0.4231
Dice	0.5260	0.5266
IoU	-	0.4496

La quasi identità tra Dice di validation (0.5260) e di test (0.5266) conferma che il modello generalizza in modo affidabile: non c'è stato overfitting nella fase di model selection, nonostante l'overfitting osservato internamente al training (Sezione 3.6).

4. Fase 2 - Estrazione degli oggetti manipolati

A partire dalla maschera (di ground truth in fase di sviluppo, predetta dal modello in fase di inferenza reale), viene calcolato un bounding box rettangolare attorno alla regione manipolata, con un padding del 10% per non tagliare bordi sfumati dell'oggetto.

4.1 Estrazione da ground truth (dataset di riferimento)

```
def get_bounding_box(binary_mask):
    ys, xs = np.where(binary_mask > 0)
    if len(xs) == 0:
        return None
    return int(xs.min()), int(ys.min()), int(xs.max()), int(ys.max())
```

Risultati dell'estrazione da maschere GT, con filtro di area minima (100 px) per scartare manipolazioni microscopiche:

Split	Oggetti estratti	Maschere vuote	Troppo scartati	piccoli
Train	3997 / 4000	0	3	
Val	500 / 500	0	0	
Test	499 / 500	0	1	

4.2 Estrazione da maschere predette (pipeline reale)

Un dettaglio tecnico fondamentale: poiché il modello di segmentazione lavora a risoluzione fissa 512x512, mentre le immagini originali hanno dimensioni variabili, la maschera predetta viene ridimensionata alla risoluzione ORIGINALE dell'immagine prima di calcolare il bounding box, per garantire coordinate corrette:

```
prob_map = torch.sigmoid(logits)[0, 0].cpu().numpy() # 512x512
prob_map_orig = cv2.resize(prob_map, (orig_w, orig_h),
                           interpolation=cv2.INTER_LINEAR)
binary_mask = (prob_map_orig > PRED_THRESHOLD).astype(np.uint8)
```

Split	Oggetti estratti	Maschere predette	vuote	Troppo scartati	piccoli
Train	3940 / 4000	45		15	
Val	485 / 500	10		5	
Test	488 / 500	11		1	

Confrontando i due risultati, la pipeline con maschere predette individua correttamente la presenza di una manipolazione nel 97-98% dei casi, con una perdita di copertura contenuta (2-3%) rispetto all'estrazione ideale da ground truth. I casi di maschera vuota predetta rappresentano i veri falsi negativi del sistema completo, e coincidono presumibilmente con le manipolazioni di area minima osservate nell'EDA (Sezione 2.2).

5. Fase 3 - Metric learning per il riconoscimento di istanze

5.1 Motivazione e scelta metodologica

Il dataset non fornisce etichette di classe/categoria per gli oggetti manipolati: l'obiettivo è determinare se due oggetti estratti rappresentano la stessa istanza fisica, non classificarli in categorie note a priori. Questo esclude approcci di classificazione supervisionata standard e richiede un approccio self-supervised di instance discrimination, sullo schema di SimCLR/MoCo:

- Ogni oggetto estratto viene trattato come la propria unica 'classe'.
- Coppia positiva: due augmentation indipendenti dello stesso oggetto.
- Coppia negativa: augmentation di oggetti diversi presenti nello stesso batch.

Le augmentation sono state scelte per simulare le trasformazioni REALISTICHE che un oggetto subisce quando riutilizzato in una manipolazione (resize, ri-compressione JPEG, aggiustamenti di colore/luminosità, flip, piccola rotazione), così che l'invarianza appresa dal modello sia coerente con lo scenario applicativo reale, non generica.

5.2 Architettura

Rete Siamese: encoder ResNet18 pretrained ImageNet (pesi indipendenti da quelli della segmentazione, per non introdurre bias task-specifici) + projection head (MLP a 2 layer, come in SimCLR) + normalizzazione L2 finale dell'embedding (necessaria per il calcolo della cosine similarity nella loss).

```
class ContrastiveModel(nn.Module):
    def __init__(self, embedding_dim=128, hidden_dim=512, pretrained=True):
        super().__init__()
        backbone = resnet18(weights=ResNet18_Weights.IMAGENET1K_V1 if pretrained else None)
        encoder_out_dim = backbone.fc.in_features # 512
        backbone.fc = nn.Identity()
        self.encoder = backbone
        self.projection_head = ProjectionHead(encoder_out_dim, hidden_dim, embedding_dim)

    def forward(self, x):
        features = self.encoder(x)
        embedding = self.projection_head(features)
        return F.normalize(embedding, dim=1) # L2-normalizzazione
```

5.3 Loss: NT-Xent (contrastive, stile SimCLR)

```
def nt_xent_loss(z_a, z_b, temperature):
    batch_size = z_a.size(0)
    z = torch.cat([z_a, z_b], dim=0)          # (2B, D)
    sim_matrix = torch.matmul(z, z.T) / temperature  # (2B, 2B)
    self_mask = torch.eye(2*batch_size, dtype=torch.bool)
    sim_matrix.masked_fill_(self_mask, float('-inf'))
    positive_indices = (torch.arange(2*batch_size) + batch_size) % (2*batch_size)
    loss = F.cross_entropy(sim_matrix, positive_indices)
    accuracy = (sim_matrix.argmax(dim=1) == positive_indices).float().mean()
    return loss, accuracy
```

Oltre alla loss, e' stata definita una metrica di monitoraggio interpretabile, la matching accuracy: la percentuale di volte in cui, dato un embedding, il modello identifica correttamente la sua vera coppia positiva come la piu' simile fra tutti gli altri elementi del batch.

5.4 Difficolta' tecnica: batch size e potenza di calcolo

Problema riscontrato

Nella NT-Xent, la dimensione del batch determina direttamente il numero di negativi disponibili per ogni ancora ($2*B - 2$). Per limiti hardware, il batch size è stato inizialmente ridotto a 8, riducendo i negativi disponibili a soli 14 per campione: un compito troppo facile, che produce una matching accuracy ingannevolmente alta senza che il modello abbia realmente imparato un buon spazio di embedding.

Soluzione adottata: attivazione del mixed precision training (AMP, autocast + GradScaler), non presente nell'implementazione iniziale, che ha permesso di riportare il batch size a 128 (254 negativi per ancora) mantenendo tempi di training sostenibili sulla GPU disponibile.

5.5 Risultati del training

Epoca	Train Loss	Train Match. Acc.	Val Loss	Val Match. Acc.	LR
1	2.0386	0.9543	1.9465	0.9661	3.00e-4
2	1.5544	0.9893	1.7559	0.9870	3.00e-4
12	1.2518	0.9996	1.5034	0.9948	3.00e-4
16	1.2266	0.9990	1.4913	0.9922	1.50e-4 (ridotto)
17	1.2160	0.9986	1.4563	0.9922	1.50e-4

La matching accuracy satura rapidamente sopra il 98-99% (atteso: l'encoder pretrained ImageNet distingue già bene oggetti visivamente molto diversi tra loro all'interno di un batch). La loss, però, continua a scendere in modo monotono, segno che il modello continua a rifinire la separazione degli embedding anche quando l'accuracy sembra già satura - la loss è quindi la metrica più informativa in questa fase avanzata del training.

5.6 Validazione qualitativa dello spazio di embedding

Per verificare la qualità dello spazio di embedding appreso, sono stati generati due controlli visivi: uno scatter t-SNE (view aumentate multiple dello stesso oggetto colorate uguali, per verificarne il raggruppamento) e un istogramma di similarità coseno tra coppie vere (stesso oggetto, 2 view diverse) e coppie casuali (oggetti diversi).

Tipo di coppia	Similarità media	Deviazione standard
Coppie vere (stesso oggetto)	0.9371	0.0639
Coppie casuali (oggetti diversi)	-0.0005	0.1207

La separazione tra le due distribuzioni è netta (medie a 0.94 e ~ 0 , sovrapposizione minima), confermando che il modello ha appreso una rappresentazione affidabile per il matching tramite soglia di similarità.

6. Applicazione: ricerca di raggruppamenti nell'intero dataset

Utilizzando il modello di metric learning, e' stato calcolato un embedding deterministico (senza augmentation casuale) per ciascuno dei ~5000 oggetti estratti, confrontandoli tutti contro tutti per individuare istanze duplicate o riutilizzate nel dataset.

6.1 Primo tentativo: DBSCAN con soglia 0.7 - problema di chaining

Risultato problematico

Con DBSCAN (metrica cosine, $\text{eps} = 1 - 0.7 = 0.3$) sono emersi due cluster anomali: il Cluster 2 con 981 oggetti e il Cluster 66 con 930 oggetti, quest'ultimo corrispondente quasi esattamente a tutti i file del sottoinsieme di face-morphing/face-swap del dataset - centinaia di persone fisicamente diverse raggruppate insieme.

Diagnosi: DBSCAN soffre del cosiddetto effetto di “chaining” (tipico del single-linkage): se A è simile a B, e B è simile a C, l'algoritmo li unisce in un unico cluster anche se A e C sono completamente diversi. Con quasi 5000 oggetti e una soglia permissiva, si formano facilmente catene di transitività che collassano centinaia di oggetti scorrelati (specialmente per categorie visivamente generiche, come volti umani ravvicinati).

6.2 Iterazioni successive

- Innalzamento della soglia di similarità da 0.7 a 0.9 (più vicina al valore osservato per le coppie vere nell'istogramma) mantenendo DBSCAN: risultato notevolmente migliorato, 163 cluster coerenti, il più numeroso con 15 oggetti, 91.3% degli oggetti senza match (plausibile data l'eterogeneità del dataset).
- Tentativo con Agglomerative Clustering a linkage 'complete' (nessun punto può entrare in un cluster se anche solo una coppia al suo interno è sotto soglia): risultato eccessivamente frammentato, gruppi genuini spezzati per una singola coppia borderline sotto soglia.
- Soluzione adottata: ripristino di DBSCAN con soglia 0.9, che a questo livello di severità non presenta più un rischio significativo di chaining (richiederebbe una catena di punti quasi identici, evento raro a soglia così stretta).

Questo percorso iterativo illustra un principio importante nel model selection per il clustering non supervisionato: l'algoritmo e la soglia non sono scelte indipendenti, e vanno validate insieme, con controllo visivo diretto dei risultati (griglie di immagini per i cluster più numerosi) piuttosto che affidandosi solo alle metriche aggregate.

6.3 Persistenza della galleria di embedding

Gli embedding calcolati su tutto il dataset vengono salvati su disco (gallery_embeddings.npy + gallery_metadata.csv con filename, split, path e cluster_id), costituendo una galleria di riferimento riutilizzabile dalla pipeline di inferenza senza dover ricalcolare tutto da zero a ogni esecuzione.

7. Pipeline di inferenza end-to-end

Lo script finale della pipeline unisce tutti e tre i modelli in un flusso unico, applicabile a una nuova immagine mai vista:

1. Il modello di segmentazione predice la maschera dell'area manipolata.
2. La maschera viene riportata alla risoluzione originale, da cui si calcola il bounding box con padding.
3. L'oggetto ritagliato viene passato al modello di metric learning per ottenere l'embedding.
4. L'embedding viene confrontato (similarità coseno) con l'intera galleria di riferimento.
5. Se la similarità migliore supera la soglia (0.85, tarata sull'istogramma di Sezione 5.6), l'oggetto viene segnalato come 'match trovato', con indicazione del cluster corrispondente; altrimenti viene considerato un'istanza unica del dataset.

Il risultato viene visualizzato in un'unica immagine riassuntiva: immagine originale, maschera predetta, oggetto estratto, e i primi 5 oggetti più simili trovati nella galleria con il relativo punteggio di similarità.

7.1 Guida pratica: come eseguire l'inferenza su una nuova immagine

Di seguito i passaggi necessari per usare la pipeline già allenata su un'immagine qualsiasi, partendo da un ambiente in cui i modelli sono già stati addestrati (checkpoint presenti in checkpoints/).

Passo 1 - Generare (o aggiornare) la galleria di riferimento

Va eseguito almeno una volta, e ripetuto ogni volta che il modello di metric learning viene riallenato o che cambia il set di oggetti estratti disponibili:

```
python src/cluster_objects.py
```

Lo script calcola l'embedding di tutti gli oggetti estratti nel dataset e salva la galleria di riferimento in `reports/clustering/` (`gallery_embeddings.npy` e `gallery_metadata.csv`). Senza questo passaggio, lo script di inferenza non ha nulla con cui confrontare le nuove immagini.

Passo 2 – Eseguire l'inferenza su una nuova immagine

```
python src/inference.py -image "percorso/alla/immagine.jpg"
```

Il parametro `--image` accetta il percorso (assoluto o relativo) a qualsiasi immagine in formato JPG/PNG. Non è necessario che l'immagine sia già presente nel dataset: la pipeline funziona su qualunque immagine esterna.

Passo 3 - Interpretare l'output

Lo script stampa a console un riepilogo testuale e salva una visualizzazione grafica in `reports/inference/<nome_immagine>_result.png`. L'output testuale ha la forma:

```
Area manipolata rilevata: bbox=(x_min, y_min, x_max, y_max)

Miglior corrispondenza: <nome_file>.png (similarita': 0.XXXX)
MATCH TROVATO -> l'oggetto sembra corrispondere al cluster <id>
(oppure)
NESSUN MATCH -> l'oggetto sembra unico, non risulta simile a nulla nella galleria
```

- Se non viene rilevata alcuna area manipolata (maschera predetta completamente vuota), lo script lo segnala e si interrompe senza produrre un confronto.
- La visualizzazione grafica mostra, da sinistra a destra: l'immagine originale, la maschera predetta, l'oggetto ritagliato, e i 5 oggetti più simili della galleria (in verde quelli sopra la soglia di match, in grigio gli altri).

Parametri configurabili

In cima al file `inference.py` si possono modificare, se necessario:

- `SEGMENTATION_CHECKPOINT / METRIC_CHECKPOINT`: percorso ai pesi dei due modelli, se si vogliono usare checkpoint diversi da quelli di default (es. il modello di segmentazione senza fine-tuning).
- `MATCH_SIMILARITY_THRESHOLD` (default 0.85): soglia di similarità oltre la quale un oggetto viene considerato un match; può essere resa più o meno permissiva a seconda del caso d'uso.
- `TOP_K_MATCHES` (default 5): quanti oggetti simili mostrare nella visualizzazione.

8. Conclusioni, limiti e sviluppi futuri

8.1 Risultati raggiunti

- Modello di segmentazione funzionante, con Dice 0.5266 e IoU 0.4496 sul test set, e generalizzazione confermata (val e test quasi identici).
- Pipeline di estrazione oggetti robusta anche con maschere imperfette (perdita di copertura contenuta al 2-3% rispetto al caso ideale).
- Modello di metric learning con separazione netta tra coppie vere e casuali (similarità media 0.94 vs ~ 0).
- Sistema di clustering che individua raggruppamenti coerenti nell'intero dataset, dopo un percorso di validazione e correzione di un problema metodologico (chaining di DBSCAN).
- Pipeline di inferenza end-to-end funzionante su nuove immagini.

8.2 Limiti riscontrati

- Il Dice di segmentazione (~ 0.53) è modesto: margine di miglioramento tramite backbone più capienti, più epoche di fine-tuning con regolarizzazione più forte, o loss aggiuntive (es. Focal/Tversky).
- Overfitting osservato nella seconda fase di training della segmentazione (train Dice 0.66 vs val Dice 0.51): il modello finale è stato scelto correttamente in base al validation, ma la capacità aggiuntiva dell'encoder sbloccato non è stata sfruttata appieno.
- Il sistema di instance discrimination riconosce invarianza solo rispetto alle trasformazioni incluse nella pipeline di augmentation: non generalizza a variazioni non rappresentate in training (es. angolazioni radicalmente diverse).
- La soglia di similarità per il matching (0.85-0.9) è stata tarata empiricamente sull'istogramma delle coppie vere/casuali: un lavoro futuro potrebbe validarla su un set di coppie annotate manualmente.

8.3 Sviluppi futuri

- Sperimentare encoder più capienti (ResNet34/50, EfficientNet) per la segmentazione, con confronto sistematico delle metriche.
- Applicare tecniche di regolarizzazione aggiuntive (dropout nel decoder, augmentation più aggressive) per ridurre l'overfitting osservato nel fine-tuning.
- Validare quantitativamente la qualità del clustering con un piccolo set di coppie annotate manualmente come ground truth per il matching.